History Consistency for Interruptible Voice Agents

OpenBot

***Abstract*—A voice agent can finish making an answer before a person finishes hearing it. If the person interrupts, generated text can remain in history even though its sound never left the player. Later answers may then assume facts were shared when they were not. This design and analysis paper defines a transcript-first baseline and derives failures from its event rules. We then propose a playback receipt and commit frontier method. The method keeps generated content, reported playback, evidence of understanding, and outside tool effects apart. It repairs both provider context and the host's view of spoken history after interruption. Worked traces and conditional proof arguments show what the method can establish. Playback reports support a claim about device output, not human hearing or understanding. Tool cancellation does not reverse completed effects. A finite model checks all 308 legal orders of an eight-event interruption and 56 clock worlds. It exposes failures when the repair barrier, epoch, or clock error bound is removed. We prove a limit on what a host can know without device evidence. Device and user studies remain future work; the model results are not measured speech performance.**

***Index Terms*—Voice agents, Dialogue history, Interruption, Playback receipts**

I. Introduction

A voice agent has several versions of an answer at the same time. The model has made some words. A server has sent some sound. A client has put some sound in a queue. A device has played a smaller part. The person may have heard and understood less still. A text history that joins these stages into one completed message can give the next answer a false starting point.

Consider an agent that says, "The room is booked. The door code is seven four two." The booking tool has succeeded. The person interrupts after the first sentence with "Which room?" The model may already have made the entire answer. Saving that text as shared history creates two different errors. The agent may assume the person knows the code. Or an overbroad repair may remove the whole answer and lose the true fact that the booking took place. Stopping sound and stopping a tool are different operations.

We use **history consistency** to mean agreement between a history claim and the evidence that supports it. A provider can keep audio while a host keeps text, and both can be consistent if each marks the same known boundary. This is narrower than actual common ground, the information people treat as shared. A system cannot infer understanding from a successful network write.

We study agents that run speech through ordinary work turns. A work turn is a unit of agent work that can use instructions, tools, and approval rules. Its end need not equal the end of audible speech. The live path must keep this account of work while responding to barge-in, meaning speech that starts during the agent's answer. Pending notices add another source of information. A tool may finish and place a notice in the conversation while the person is speaking. Acceptance of that notice does not mean its contents were spoken.

This paper makes three contributions. First, it gives an evidence model that separates generated, transmitted, played, understood, and externally effected content. Second, it proposes a per-item commit frontier and interruption barrier that connect client playback to both provider context and ordinary history. Third, it gives a limit on receipt-free knowledge, a conditional maximum safe prefix, and an executable finite comparison with separate device and host state. These are protocol and finite-model results. The full receipt method is not claimed as a deployed voice feature. General crash recovery, tool retry policy, and distributed ownership are outside the paper's main scope.

II. Grounding and Incremental Dialogue

Clark and Schaefer describe a contribution to dialogue as joint work that goes beyond producing an utterance. The speaker and addressee need evidence of enough understanding for their purpose [1]. This supports a strict distinction in our model: putting sound on a device is evidence of presentation, not proof of a completed contribution. A playback receipt should therefore never be named a receipt of understanding.

Clark and Brennan show that grounding depends on the task and communication medium [2]. A brief acknowledgment may be enough for a casual update. A code or action with a large cost may need a repeat or a clear confirmation. Our design retains this distinction through a separate grounding field. It does not demand a verbal confirmation after every sentence. Instead, it keeps the reason for a belief available, so policy can use a stronger standard when needed.

Schlangen and Skantze model incremental dialogue as connected units that can be added, revoked, and committed [3]. That work provides a close conceptual base. Our commit frontier also separates tentative from committed content. The difference is its boundary: a playback device, a provider conversation, and a host work turn do not share a single state machine. We connect their evidence without treating a provider's completed output as an irreversible conversational contribution.

Skantze and Hjalmarsson study speech generation that plans, realizes, and monitors output while input continues [4]. Their work makes self-monitoring central rather than waiting for whole turns to end. Skantze and Schlangen also study an incremental number dictation system that responds to listener feedback [5]. These are design precedents; their measured findings do not establish this protocol's performance.

The OpenAI Realtime client-event reference describes a request to cut an assistant audio item at a stated time. The server removes its text transcript and returns a success event

when the cut succeeds [6]. This gives a provider context operation. It does not give a precise shortened host transcript, a device receipt, or evidence of understanding. Our adapter therefore treats provider repair and host projection as separate obligations. We checked this reference on 27 October 2026; a provider version must be pinned before a live test.

A device clock is also an interface with stated limits. The W3C Web Audio Recommendation separates the output stream position from the context processing clock. Its output timestamp relates a sample frame to an estimated real-time moment [7]. We do not assume a browser clock proves sound reached a person. The method requires an explicit bound on clock overstatement before it treats a reported position as a lower bound on rendered audio.

The distinction from prior incremental dialogue work is thus narrow. Add, revoke, and commit are established ideas. This paper studies the evidence needed to join a separate playback device, provider context, and saved tool outcome. It does not claim a new theory of grounding or a faster speech generator. The new proof obligations concern that join.

III. A Transcript-First Baseline

We define a baseline, B, rather than claim that a particular service uses it. B has a host, a speech provider, and a player. The host owns an ordered text history, a queue of pending inputs, and at most one active work turn. The provider owns its own conversation context. The player owns a first-in, first-out audio queue. Each actor processes its received events in order. Messages between actors can be delayed independently. Clocks need not be shared.

The provider sends six event forms. `InputFinal(u, text)` identifies a completed input transcript. `Words(i, text)` appends generated transcript text to output item i. `Sound(i, bytes)` appends generated sound. `ItemEnd(i)` ends generation for that item. `Call(c, args)` asks for a tool action. `ResponseEnd(status)` ends one model response. Here u, i, and c are distinct input, output, and call identities. None of these events means that sound has played. A separate `SpeechStart` event reports detected user speech.

The host handles a final, nonempty transcript by appending a user input to its pending queue. It marks that input as already present in provider context, so it does not send the transcript back as a second user message. This mark is keyed by u and consumed once. Typed input has a fresh identity and is sent normally, even when its words match earlier speech. Recognition errors can still remain in a final transcript.

When no work turn is active, the host removes the next pending input, appends it to history, and starts a work turn. Automatic provider answers are disabled. The host requests each response with instructions and allowed tools. On `Call(c, args)`, it checks approval before dispatching c. A completed result is saved with c and returned to the provider once within this connection. Tool success is an outside fact; text describing that success is a separate generated item.

For every `Words` event, B appends the text both to its output record and to history used by later turns. For every `Sound` event, B sends bytes to the player in receive order. This baseline transport does not attach item or sample metadata to those bytes. Text and item-end events retain their identifiers. The player queues and renders bytes but sends no playback receipt. When a response ends with no tool continuation, the host marks its work turn ended and clears its active flag. This occurs without waiting for the player's queue to empty.

B handles `SpeechStart` or an explicit stop request as follows: if a work turn is active, it sends a player stop command, cancels provider generation, and ends the work turn. If no work turn is active, it sends no stop command. The player drops queued audio when it receives a stop. Cancellation does not modify an already saved tool outcome. These exact rules define the generation-guarded stop baseline used in our counterexample.

For context repair, define two variants. B0 retains all generated text. B2 also accepts a client-supplied item and cut offset and forwards a provider truncation request. B2 does not revise host history and does not wait for truncation confirmation before requesting the next response. Thus even a correct client cut repairs only one of the two context stores. A separate comparison B1 deletes the entire interrupted output item from both stores. Its cost is loss of any supported prefix.

On a new connection, each baseline seeds provider context from its host history. A notice from a tool or another permitted input source enters the same pending queue as user input, with its source identity attached. Queue acceptance and later work-turn completion are recorded, but the baselines have no presentation state for the notice. These rules yield three failures: generated text can outlive its sound, an ended response can leave audio playing, and history seeding can undo provider-only repair.

IV. An Evidence and Time Model

Let an output item have identity i within output epoch e. An epoch is a fresh name for one answer's playback lifetime; one connection can span many epochs. It prevents old events from being mistaken for new output. Audio offsets use the item's sample clock, not the host wall clock. For a contiguous item, let $G_i(t)$ be generated audio duration, $S_i(t)$ be duration sent, and $P_i(t)$ be duration rendered by the playback device at time t. Let $A_i(t)$ be the greatest valid playback offset known by the host.

Under honest receipts, ordered contiguous playback, and a correct sample map, the intended relation is

$$0 \le A_i(t) \le P_i(t) \le S_i(t) \le G_i(t). \tag{1}$$

The relation is about one item, not total wall time. Repeated audio and skipped ranges need a range set rather than a single number. Our first method excludes those cases within an item. A replay is a new item that refers to the old one. Lost or discarded output breaks the playable prefix and cannot advance a cumulative receipt past the gap.

Human hearing is a separate latent variable. A muted device, noise, divided attention, or a hearing difference can make rendered sound unavailable to a person. There is no justified equality between P_i and heard content. Understanding

is still another state. The system records positive or negative evidence of understanding with the relevant proposition, such as a clear repeat, a correction, or an explicit confirmation. Lack of a complaint is not recorded as certainty.

Each text unit has a span in the generated transcript and, when available, a conservative audio interval. Unit boundaries and semantic completeness are inputs to this protocol; the protocol does not infer them correctly by construction. A conservative interval contains the sound needed for that unit. Let b_j be its known end, and let m_j mean that its mapping is reliable and its meaning is complete enough to stand as a claim. The commit frontier is the largest consecutive set of units whose end offsets satisfy $b_j \leq A_i$ and whose m_j values hold. Unknown mappings stop the frontier. A partial clause stays in the evidence record but does not become an asserted fact. Even a complete rendered unit may be false. The proved property is faithful presentation evidence, not truth of the words or their meaning to the listener.

TABLE I
DIFFERENT EVIDENCE SUPPORTS DIFFERENT HISTORY CLAIMS. NO ROW IMPLIES ALL LATER ROWS.

Evidence	Permitted statement
Generated text	The model made these words.
Socket write	The host sent these bytes.
Playback receipt	The device reports this prefix played.
Grounding evidence	The person gave this response to the claim.
Tool result	The tool reports this outside outcome.

A history record therefore has three projections, meaning three views made from the same record. The audit view contains generated words, cut points, and later evidence. The presentation view contains supported spoken units and explicit uncertainty for incomplete output. The effect view contains tool outcomes whether or not their reports were spoken. Grounding evidence qualifies the presentation view; it is not inferred from the effect view.

A pending notice has at least an arrival event, queue acceptance, eligibility for speech, and any later presentation evidence. These are separate times. An input policy checks the notice's source and grants or rejects acceptance before queue insertion. We study only how its content enters the next spoken exchange. One local serialization point orders interruption decisions and the next answer. This assumption does not claim a global ownership solution across servers.

V. PROPOSED PLAYBACK FRONTIER METHOD

A. Name output and report playback

The proposed transport adds an explicit association between each audio range and its output epoch, item, content part, and sample offsets. A binary envelope or a preceding range descriptor can carry that association. The client must not guess it from a nearby transcript delta. The same identity links the provider item, generated text record, and work turn. This is an addition to the defined baseline transport.

The player reports the largest contiguous range that has crossed its selected output boundary. The implementation must define that boundary. A callback for placing sound into a software queue is insufficient. A device render clock, with measured uncertainty, is a better source. If the available clock gives only queued position, the receipt is labeled queued and cannot advance the played frontier. Every receipt contains an epoch, item, offset, and increasing receipt number. It also declares whether playback is active or stopped.

The host rejects unknown items, wrong epoch-item pairs, negative offsets, offsets beyond the sent range, and incompatible clocks. Duplicate receipts have no effect. Reordered older receipts do not reduce the frontier. A later valid receipt can advance the known prefix even after generation ends. This allows an ended work turn's remaining audio to finish without making a work-turn end event stand for playback completion.

If a receipt reports sample position q with an overstatement bound ε, its safe lower bound is $\max(0, q - \varepsilon)$. The host takes the maximum of all accepted lower bounds for that item. This maximum is still safe because each bound concerns the same contiguous playback. An unknown error bound supplies no positive guarantee. A queued-only report cannot be repaired by choosing an arbitrary small error. After a final stop receipt closes an item, later reports may repeat its final bound but cannot increase it.

B. Build a supported transcript

The host stores generated transcript units separately from their presentation status. If reliable word or phrase timing exists, it commits only complete supported units. It does not estimate text length by multiplying character count by played duration. Speech rates differ, pauses consume time, and a small missing word can reverse the meaning of a sentence. For "Do not send the file," a cut before "not" is not evidence that a positive send instruction was conveyed.

When exact alignment is missing, the first version uses a coarser complete-item rule. It keeps an interrupted item's text as tentative, with a record that some audio was played. It adds no exact shortened quotation. If full-item playback and a valid transcript are both established, it may mark the whole item presented. A future aligner can improve recall, but its error rate must be evaluated. The method favors an explicit gap over a fabricated exact prefix.

The model's next prompt gets the presentation and effect views with their labels. A policy might say that a booking succeeded but its room name has no supported spoken report. The agent can then answer "It is room three" without saying "as I told you." This lets a policy avoid treating unplayed content as shared while retaining true outside results.

C. Reconcile an interruption

Speech start retires the output epoch for new scheduling. Each actor applies retirement when it processes the stop event; this cannot retract sound already in flight. The client stops its queue and sends a final stop receipt. The host cancels generation and ends the work turn while preserving saved tool outcomes. A call received after retirement is recorded but not

newly dispatched. New user audio continues to arrive during repair.

The stop receipt closes the item's playable prefix. The host records a repair key with the provider session, output epoch, item, content part, request identity, and chosen cut. It sends provider truncation for that cut and waits for success tied to that key or a known failure. A late success from an old session cannot open a new barrier. Where the provider does not echo a request identity, the adapter must serialize repairs for that item and bind the received event to the one pending request. A bare item name and offset are not enough across sessions. Late output for the retired epoch remains audit data and is not scheduled. A late final receipt for that epoch is still accepted when it reports sound already rendered before the stop. Retiring an epoch does not mean ignoring evidence about its past.

Before starting the next provider response, a barrier requires the affected provider context and the host presentation view to use compatible boundaries. A barrier is a short ordering rule, not a pause in recording. It can release after stop evidence and provider repair are resolved. If they cannot be resolved within the chosen deadline, the host may replace the provider session with one built from conservative history, or send a fixed repeat request while withholding provider responses. It must not treat the generated suffix as shared. Session replacement here is live context repair, not a general crash recovery scheme.

TABLE II
THE NEXT ANSWER WAITS FOR CONTEXT REPAIR; MICROPHONE CAPTURE DOES NOT.

Step	Proposed state change
Generate	Append tentative text and named audio ranges.
Receipt	Advance only the valid played prefix.
Barge-in	Retire output scheduling; keep new input open.
Stop	Accept final played offset; request truncation.
Repair	Match provider boundary and host view.
Respond	Use supported speech and separate tool facts.

D. Preserve tools and pending notices

A tool call has its own outcome: not started, running, succeeded, failed, or unknown. A call begins as not started. Dispatch makes it running; a confirmed result makes it succeeded or failed. Losing the result channel before confirmation makes a running call unknown. Stopping a work turn can prevent later work or request cancellation of work already started. It cannot make a completed booking cease to exist. An unknown result stays unknown until the tool's own evidence resolves it.

A notice about a completed action retains its source identity when it enters speech. Marking it accepted into the queue is distinct from marking its content presented. If its report is cut off, the next work turn can report the missing result once. This does not rerun the underlying action. A user correction and a pending notice are both included with their causal order. The method avoids creating a hidden voice queue that can ignore ordinary approval and work-turn rules.

VI. Worked Interruption Traces

All times in this section are illustrative. They are chosen to show state transitions, not measured performance. In the first trace, a provider makes 4.0 seconds of audio by host time 0.7 seconds. The host sends all 4.0 seconds by time 0.9. The device begins at time 0.2. The person interrupts at time 1.8, and the device stops after rendering 1.7 seconds. The latest earlier receipt covered 1.5 seconds.

At the first stop decision, $A_i = 1.5$, while the final device evidence will support $P_i = 1.7$. The generated but unplayed amount is $4.0 - 1.7 = 2.3$ seconds. Sent duration gives the wrong cut even though transmission succeeded. The final receipt advances the host frontier to 1.7 seconds. A reliable phrase map places the end of "The room is booked" at 1.4 seconds. The code begins at 1.9 seconds. Only the first statement enters the presented view. Provider truncation uses 1.7 seconds. The tool result still records the booked room and code.

Now suppose the generated response finished and the work turn ended at time 0.8. The same audio remains in the player until 4.2. An interruption at 1.8 is still real even though no model answer is running. The proposed playback owner handles it. By B's stop rule, its active flag is false at 1.8, so it sends no stop. The remaining sound continues. This is a counterexample derived from B's rules, not an observed deployment failure. Removing the active guard alone stops sound but does not repair B0′s full transcript or B2′s retained host history.

In the second trace, a booking tool reports success at 2.0 seconds. A notice is accepted at 2.1, and speech begins at 2.4. The person starts a correction at 2.5 before the phrase "The booking is complete" is fully played. The system has evidence of success but lacks a completed spoken report. The next answer must check the correction against the existing booking. It may report that the booking is already made and discuss a change. It must not claim cancellation removed the booking, and must not submit the same booking again merely because its first report was interrupted.

A third trace has a delayed stop receipt. The host knows only 0.6 seconds played when the person asks "What was the last part?" A receipt for 1.1 seconds arrives after a newer session has opened. Its old epoch prevents it from changing the new player's state. It may still refine the old audit record and its presentation view if its identity is valid. Until that evidence arrives, the next answer uses the conservative boundary or offers a repeat. The cost is possible repetition of already played words. The evaluation measures this repetition cost.

VII. Knowledge Bound and Consistency Properties

Proposition 1: a host without playback evidence cannot recover an exact nonempty played prefix. Let two runs have identical generated text, sent audio, tool results, and all messages received by the host. In run A the device is stalled before the first sample. In run B it has played the whole item. These runs are allowed when device delay is unbounded

and no playback evidence is sent. Any host rule receives the same observation in both runs and must make the same commitment. Every positive commitment is unsound in A; an empty commitment loses supported content in B. Therefore no such rule is both sound and complete for all allowed runs. This argument also covers randomized rules that demand zero error in every run. Network delivery or provider completion does not remove the indistinguishable pair.

Proposition 2: exact boundaries give the greatest sound eligible prefix. Fix one item, its ordered unit map, and its receipt history. For this result, assume that each mapped end b_j is the exact required completion boundary, not merely an upper bound: a unit is incomplete when playback ends before b_j. Let W be all device histories consistent with the available evidence. Sample offsets are integers in a finite sent range, so a least possible played position L exists and is attained by some history in W. Among consecutive prefixes made only of eligible units, the frontier is the greatest prefix supported in every history in W.

Proof. Each retained unit ends at or before L, so it is complete in every possible history. A longer eligible prefix includes a unit whose required end is greater than L. The history attaining L leaves that unit incomplete, so the longer prefix is not sound for all possible histories. With only sent length and honest exact receipts, L is the largest receipt offset. With the stated one-sided error bounds, it is the largest accepted lower bound. These equalities assume no further timing evidence; a stronger measured device guarantee could shrink W.

With merely conservative upper ends, the frontier remains sound but need not be greatest among all sound prefixes. For example, a word may end before an upper interval that also includes silence. Such a map can lose a word that was fully rendered. We claim only the greatest prefix allowed by the conservative rule in that case. An unknown early map can also block later units. A non-prefix view could preserve some later independent units but would need a different result and a rule for their meaning out of order.

Presentation soundness. Assume honest device receipts, correct range identity, no gaps in acknowledged playback, and conservative text alignment. Every committed text unit was rendered according to the chosen device boundary. The proof is by induction over accepted events. Generation and sending add no committed unit. A receipt advances only to a validated played offset. The commit rule selects units ending at or before that offset. Stop events and late data do not bypass the rule. This proves an evidence property, not hearing or understanding.

No return of an unsupported suffix. Assume the next answer cannot start before the interruption barrier, and every prompt construction path uses the repaired presentation view. Then a cut generated suffix cannot reappear as an established spoken contribution solely through local history replay. The barrier first removes or marks it in provider context. The host projection does the same when a session is rebuilt. Both context paths preserve the cut. This is a property of supplied context, not a guarantee about every statement a model may generate. A path that reloads raw generated text as established assistant speech violates the assumptions.

Effect preservation. If a tool's confirmed outcome is held in the effect view, any sequence of playback stops, transcript cuts, and speech receipts leaves that outcome unchanged. This follows because none of those transitions writes the tool outcome. A later explicit reversal is a new tool action with its own evidence. Keeping it separate prevents conversational editing from pretending to undo outside work.

Bounded evidence lag. Suppose receipts are emitted at most every r seconds, transport and processing take at most d, and playback clocks are exact. During steady contiguous playback at normal speed, the host's known audio prefix lags by at most $r + d$ seconds. For a textual bound, assume every unit is semantically eligible, units cover the stream in order with length at most u, and each reliable map reaches the host within l seconds after its unit finishes playing. Both receipt and map are then available for units ending by $t - \max(r + d, l)$. The last such complete unit ends at most u seconds earlier. Textual lag is therefore at most $\max(r + d, l) + u$. A single permanently unmapped or ineligible early unit breaks this premise and can block the frontier forever. No finite textual lag is claimed in that case.

Progress needs separate assumptions. If final stop evidence and provider repair arrive, and ordinary scheduling is fair, the barrier can release. Without that evidence the method needs its declared fallback. It cannot guarantee both immediate next-turn speech and exact retained context under arbitrary delay. This is a live coordination tradeoff rather than an exactly-once execution claim.

VIII. Remaining Device and User Evaluation

The finite tests below have run. Device timing, speech alignment, and the spoken task study have not. We propose a device event harness followed by a limited spoken task study. The harness uses scripted provider events, a controlled playback clock, work-turn inputs, and fake tools with explicit external outcomes. It records generation, send, render, receipt, truncation, and next-response events. This gives a known device-output ground truth without presenting it as human understanding.

The task set includes information lists, codes, a booking with a read-only lookup, and a simulated state-changing tool. Each task has content units and known dependency relations. Cuts occur before a key word, within a negation, between clauses, after generation completes, during a tool call, and during a notice report. Repeated user words test duplicate suppression. Delayed receipts, reordered receipts, slow playback, transcript lag, and truncation failure test the boundaries. Use synthetic audio for exact timing tests and separately recorded speech for alignment tests.

Compare B0, B1, B2, and the proposed method under the same generated outputs. For history comparisons, give all four the same player stop timing; test the generation guard separately. B0 retains generated text, B1 removes the whole interrupted item, and B2 repairs provider context alone. The proposed method uses a shared frontier and separate effect

facts. Holding model output fixed first isolates history policy. A later live-model comparison can test downstream language behavior, but must report model version and sampling settings because the response itself can change.

Measure three forms of latency separately: end of user speech to first device output, start of user interruption to final device stop, and final device stop to the next allowed response. Record median and tail values rather than one average. Also measure receipt traffic, repair wait, repeated supported content, lost supported units, and divergence between provider and host cut boundaries. Report the render boundary and its uncertainty with every timing result.

Define an unsupported-common-ground rate as the number of later references that presuppose a unit was shared without the required evidence, divided by all later references that presuppose earlier sharing. Also report the count per interrupted exchange, since a system can lower the rate by avoiding references altogether. In the harness, evidence means played units under the declared rule. In the spoken study, annotate evidence of understanding separately. Never score device output alone as demonstrated human knowledge.

Tool metrics count repeated external actions caused by a missing spoken report, confirmed results erased from working context, and unknown outcomes falsely stated as success or cancellation. Notice metrics count results never reported and results reported again after a confirmed complete presentation. These measures test both conversation and task behavior.

Ablations remove the epoch, final stop receipt, provider barrier, conservative text rule, or separate effect view one at a time. Use paired traces with the same cut and delay schedule. Report paired differences with bootstrap confidence intervals over whole dialogues, not independent audio chunks. Predefine primary metrics and comparison families. A spoken study should randomize condition order, use simulated outside actions, obtain consent for recording, and let two independent annotators mark unsupported assumptions. Report disagreement and adjudication rules. Select participant count from a pilot estimate of dialogue-level variance before the main comparison; no sample size or power result is claimed here.

IX. Finite Model Results

The artifact `papers/experiments/papers_13_14.py` runs with Python's standard library. It uses no model or audio device. Run the repository experiment entry point with 30 seeds; its JSON file records source hashes, the interpreter version, every comparison count, and first counterexample traces. Whole-fixture assertions and world counts are reported separately. Neither is a count of people, real dialogues, or independent statistical samples.

For the order test, the model starts with two sent units and one saved successful tool outcome. It enumerates all permutations of eight events: render, ordinary receipt, retire, device stop, final receipt, provider acknowledgment, late output, and answer attempt. Seven causal edges are explicit in the artifact: render precedes its ordinary receipt and stop; retire precedes stop, late output, and answer; stop precedes final receipt; final receipt precedes acknowledgment. These rules admit 308 orders. The ordinary receipt may arrive after the final receipt. The player renders only the first unit. The host state is checked against separate device-render and provider-apply events after all 2,464 event prefixes. The host routine is not used to compute the expected answer gate.

All prefixes preserve the saved tool result and avoid unsupported committed units. Of 308 answer attempts, 70 occur after valid repair and 238 are blocked. All completed schedules permit a later answer and retain the same one supported unit. Removing the barrier permits an answer in the saved trace before the provider acknowledgment. Four wrong repair fields, including epoch and request identity, are tested after each final receipt. A separate old-epoch trace shows that dropping epoch checks lets a receipt for an old item commit a new, unplayed unit with the same name.

The clock test has 15 observation classes: reported position zero through four, crossed with an overstatement bound zero through two. It enumerates all 56 possible device worlds allowed by those observations. A separately computed intersection of supported unit sets is the oracle. The lower-bound frontier equals that intersection in every class. Using the raw reported position instead would admit unsupported units in 11 of the 56 worlds. These are exhaustive counts in this small domain, with no probability model or confidence interval. They do not calibrate a real clock. A separate map fixture ends a word at sample one but gives it a conservative upper end at sample two. After one rendered sample, the conservative rule loses that word while the exact map retains it. This is the executable boundary of Proposition 2.

TABLE III
One paired two-unit fixture. Every rule receives the same generated output and device cut. Counts are synthetic units, not later model statements.

History rule	Unsupported host units	Lost supported units	Views differ
B0: keep text	1	0	No
B1: delete item	0	1	No
B2: provider only	1	0	Yes
Frontier	0	0	No

The timed fixture computes 96,000 played samples under the generation guard and 40,800 after the declared stop. The 2.3-second difference follows from supplied virtual times. The notice fixture makes one outside call under separate effect and presentation state; the retry-on-cut ablation makes two. Thirty fixed seeds add 900 receipt arrivals with duplicates, reordering, and invalid identity or range. The full paper-13 record contains 18,273 assertions in this run. Fixed finite domains, seeded stress checks, and illustrative timing each have distinct meanings; none estimates real speech latency.

X. Reproducible Fixtures and Limits

A minimal simulator needs no speech model. It uses a stable event queue ordered by scheduled time, actor identity, then insertion number. Each event records its cause and its output identity. Audio consists of numbered sample ranges. A render event consumes a range at a declared sample rate; stop

discards ranges not yet consumed. The simulator can therefore compare receipts against an exact played prefix. Text labels are synthetic units with known sample intervals. They test protocol logic, not the quality of real speech alignment.

A first fixture gives one item 96,000 samples at 24,000 samples per second. It is generated by 0.7 seconds and sent by 0.9. Playback starts at 0.2, the work turn ends at 0.8, and speech start arrives at 1.8. Without a stop, playback ends at 4.2. With a stop at 1.9, exactly 40,800 samples play: 40,800 divided by 24,000 equals 1.7 seconds. The first phrase ends at sample 33,600, and the second begins at sample 45,600. These values instantiate the first trace.

A second fixture places a confirmed tool result before a notice, then cuts the report before its complete-item boundary. Its assertions are that the outcome remains succeeded, the notice remains without a completed spoken report, and the next turn may report the result without dispatching the tool again. A third fixture delivers receipt numbers 3, 2, 3, and 4, with offsets 0.6, 0.4, 0.6, and 1.1 seconds. It checks monotonicity, duplicate handling, and separation from a new epoch. Add invalid offsets beyond the sent duration and require rejection without frontier change.

For every fixture, publish the initial state, total event order, injected delays, expected presentation units, expected tool facts, and final provider boundary. The finite explorer above covers one item, one render step, one interruption, and one provider repair. Its checks are receipt soundness, no scheduling after retirement, no response before repair, and no tool-state mutation by presentation events. Longer traces, multiple simultaneous items, and real provider failure behavior remain untested by this model. Its independent oracle shares the written specification and can still share a specification error. The counterexample ablations show sensitivity to the named guards; they do not prove the absence of every model bug.

Several limits remain. Reliable text timing may be unavailable. A complete-item rule can lose useful context and cause repetition. Alignment errors can create false support even with correct playback receipts. Device clocks can overstate audible output, and an untrusted client can lie. The protocol assumes a trusted player and exposes the evidence type rather than claiming universal assurance. The provider must support confirmed context editing or accept a new session built from an explicit prompt; otherwise the next-response barrier cannot establish its promised boundary.

A single listener and one ordered output stream are assumed. Multiple listeners need separate evidence records; one person's receipt cannot stand for another's experience. Captions need their own display evidence. Input transcription can change meaning, so waiting for a final transcript improves stability without proving correctness. Long-term storage recovery remains outside this paper's boundary.

XI. Conclusion

Interruptible voice history needs more than a completed response and a stop signal. The transcript-first baseline shows why ordinary work turns, tool results, and pending input need a separate account of presentation. The proposed method adds that account and a shared cut boundary for provider context and host history. Its conditional properties preserve supported speech without erasing real tool outcomes. Its limits are equally clear: rendered audio is not known hearing, and known hearing is not known understanding. The finite comparison now exposes failures in the named baselines and validates the bounded receipt model against separate actor evidence. The remaining step is the stated device and user evaluation of latency, retained useful context, and unsupported shared assumptions.

References

[1] H. H. Clark and E. F. Schaefer, "Contributing to Discourse," *Cognitive Science*, vol. 13, no. 2, pp. 259–294, 1989, doi: 10.1207/s15516709cog1302_7.

[2] H. H. Clark and S. E. Brennan, "Grounding in Communication," in *Perspectives on Socially Shared Cognition*, L. B. Resnick, J. M. Levine, and S. D. Teasley, Eds., American Psychological Association, 1991, pp. 127–149. doi: 10.1037/10096-006.

[3] D. Schlangen and G. Skantze, "A General, Abstract Model of Incremental Dialogue Processing," in *Proceedings of the 12th Conference of the European Chapter of the ACL*, Association for Computational Linguistics, 2009, pp. 710–718. [Online]. Available: https://aclanthology.org/E09-1081/

[4] G. Skantze and A. Hjalmarsson, "Towards Incremental Speech Generation in Dialogue Systems," in *Proceedings of the SIGDIAL 2010 Conference*, Association for Computational Linguistics, 2010, pp. 1–8. [Online]. Available: https://aclanthology.org/W10-4301/

[5] G. Skantze and D. Schlangen, "Incremental Dialogue Processing in a Micro-Domain," in *Proceedings of the 12th Conference of the European Chapter of the ACL*, Association for Computational Linguistics, 2009, pp. 745–753. [Online]. Available: https://aclanthology.org/E09-1085/

[6] OpenAI, "Realtime Client Events." Accessed: Oct. 27, 2026. [Online]. Available: https://developers.openai.com/api/reference/resources/realtime/client-events

[7] World Wide Web Consortium, "Web Audio API: W3C Recommendation, 17 June 2021." Accessed: Oct. 27, 2026. [Online]. Available: https://www.w3.org/TR/2021/REC-webaudio-20210617/